

Power-Analysis Side-Channel Attack on an Arduino-Based Digital Lock

Vincent Liu
ENGR 098 – Final Project Report

Fall 2025

Abstract

Side-channel attacks exploit physical leakages—such as power consumption, timing, or electromagnetic emissions—to extract secret information from otherwise secure cryptographic devices. This project demonstrates a proof-of-concept power-analysis attack against a simplified Arduino-based digital passkey lock. By inserting a shunt resistor into the microcontroller’s power path and capturing high-speed voltage waveforms with a digital oscilloscope, I show that different input digits generate distinct power signatures. Using Python to collect traces and perform waveform matching, the system is able to recover the secret four-digit passkey from power measurements alone. The report concludes with a discussion of limitations and practical countermeasures, including noise injection, hardware power filtering, and lockout mechanisms.

Introduction

Modern cryptographic algorithms are mathematically strong, but real-world implementations often leak information through unintended channels. Side-channel analysis focuses on these physical leakages rather than on breaking the algorithm itself. For embedded systems and Internet-of-Things (IoT) devices, power-analysis attacks are especially important because attackers may obtain physical access to the hardware.

The objective of this project is to build a complete, end-to-end demonstration of a power-analysis side-channel attack on a small embedded system. An Arduino Uno R3 is used to emulate a digital passkey lock with a fixed four-digit secret. The project follows an ethical, defensive perspective: by understanding how the attack works, system designers can better harden future designs.

Background on Power-Analysis Attacks

When a microcontroller executes instructions, its instantaneous current draw depends on internal data values, memory accesses, and control flow. If a small shunt resistor is placed in series with the power supply, these current variations appear as measurable voltage changes. A digital oscilloscope can capture these voltage waveforms with high temporal resolution.

In a typical power-analysis attack, the adversary:

1. **Measures voltage:** insert a shunt resistor into the power path and probe the voltage across it.
2. **Collects traces:** record power traces while the device performs cryptographic operations or secret-dependent tasks.
3. **Identifies leakage:** locate regions in the waveform where the power consumption depends on the secret data.
4. **Runs analysis:** apply statistical or machine-learning techniques to recover the secret.

This project demonstrates a basic, template-matching style attack rather than a full correlation power analysis (CPA) against a standard cipher. The aim is pedagogical: to clearly illustrate how different input digits generate reproducible, distinguishable power signatures.

System Overview

Arduino-Based Digital Passkey Lock

The target system is an Arduino Uno R3 acting as a simple electronic lock. The firmware stores a secret four-digit passkey internally. The device receives input digits from a host (via serial communication) and, after four digits are entered, compares the input sequence to the stored key. To better emulate realistic computation, the check routine includes small delays and arithmetic operations so that each digit produces a non-trivial waveform.

From a security perspective, the Arduino is treated as a stand-in for a generic embedded device implementing a PIN check or lightweight cryptographic routine.

Threat Model and Ethical Scope

The attacker is assumed to have physical access to the device, including the ability to insert a shunt resistor and attach an oscilloscope probe. The goal is to recover the secret passkey without modifying the firmware or reading the code directly.

All experiments were performed on my own hardware in a controlled lab environment. The purpose of this work is educational and defensive: to highlight real risks and motivate robust countermeasures in future designs.

Experimental Setup

Hardware Configuration

The measurement setup consists of:

- Arduino Uno R3 development board acting as the digital lock.
- Bench power supply providing a stable 5 V supply.

- Low-value shunt resistor inserted between the power supply and the Arduino ground rail.
- Siglent SDS1000X-E series digital storage oscilloscope for high-speed waveform capture.
- Standard oscilloscope voltage probe across the shunt resistor.
- Additional trigger connection from the Arduino to a digital channel to align traces.

Figure 1 shows the overall bench setup, including the power supply, oscilloscope, breadboard, and Arduino board. A close-up of the connection between the shunt resistor, probe, and board is shown in Figure 2.



Figure 1: Hardware setup for the power-analysis experiment. The Arduino Uno R3 is powered through a shunt resistor while the oscilloscope measures the resulting voltage waveform.

Software and Data Acquisition

To automate data collection, I used Python with the PySerial library to communicate with the Arduino over USB. For each experiment:

1. A chosen digit is transmitted from the host PC to the Arduino.

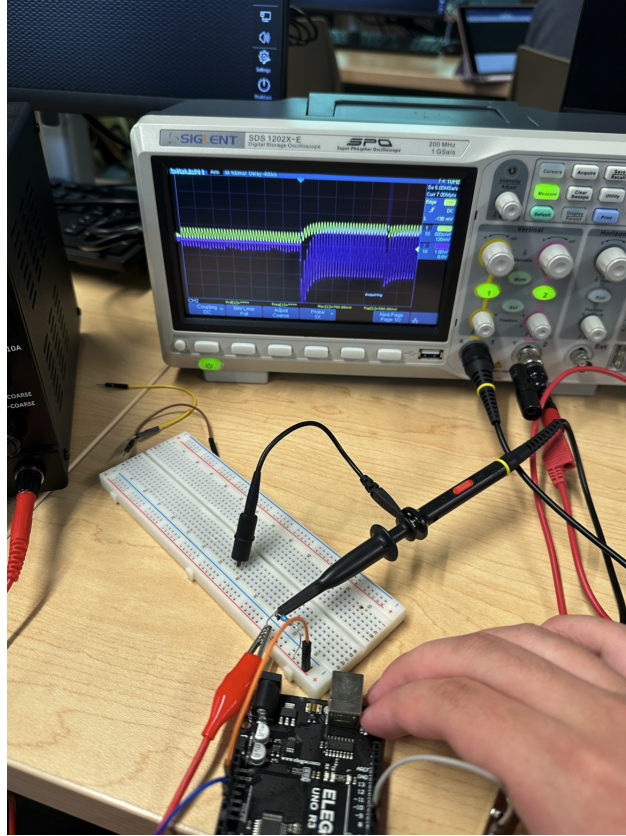


Figure 2: Close-up of the Arduino and breadboard connections. The oscilloscope probe measures the voltage across the shunt resistor while a trigger signal aligns traces.

2. The Arduino processes the digit and toggles a dedicated trigger pin at the start of computation.
3. The oscilloscope is configured for single-shot acquisition, triggered by the digital signal.
4. The resulting waveform is captured and transferred to the PC for offline analysis.

The process is repeated many times for each digit to build a set of representative power traces. The oscilloscope sampling rate is chosen so that several hundred to over one thousand samples cover the time interval where the digit is processed.

Power Trace Collection and Visualization

To analyze the passkey, the full trace is segmented based on the trigger timing and known delays between digit checks. Figure 3 illustrates the same waveform divided into four regions, each corresponding to one digit position in the secret key.

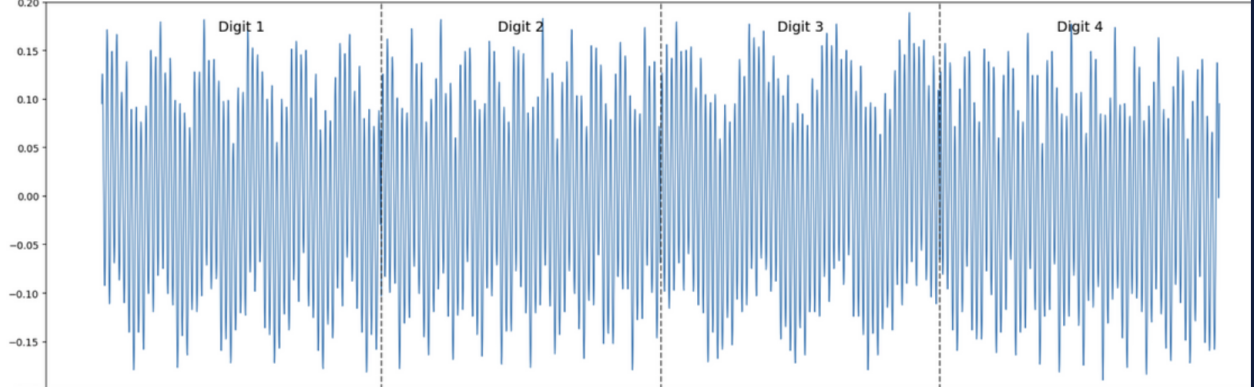


Figure 3: Segmented power trace showing distinct regions for the four digits. Each segment is processed independently to infer the corresponding digit.

Digit Templates and Classification

Template Collection for Digits 0–9

To build a classifier, the system first enters a *training* phase in which the Arduino is instructed to process controlled input digits from 0 to 9. For each digit, multiple traces are captured, aligned using the trigger, and averaged or smoothed to form a characteristic waveform.

Figure 4 shows one representative trace per digit. Even to the naked eye, the overall envelope and local oscillations differ slightly between digits.

Template Matching for Unknown Digits

In the *attack* phase, the system records power traces for an unknown four-digit passkey. For each digit position, the captured segment is compared against the ten template waveforms.

In this project, classification is performed in the time domain using straightforward waveform similarity measures. A nearest-neighbor approach, such as Euclidean distance or correlation between the unknown trace and each template, is sufficient to distinguish digits reliably for the simplified lock implementation.

Figure 5 compares each of the four unknown-digit segments (blue curves) with the best-matching template trace (orange curves). The high degree of overlap indicates that the classifier is correctly identifying the digit.

Results

After the training phase, the classifier is applied to a sequence of traces in which the Arduino checks its secret passkey. For each of the four digit positions, the classifier outputs the index of the most similar template waveform.

In the final experiment, the system successfully recovered the Arduino’s secret PIN as

$$\text{PIN} = 3 \rightarrow 7 \rightarrow 1 \rightarrow 9.$$

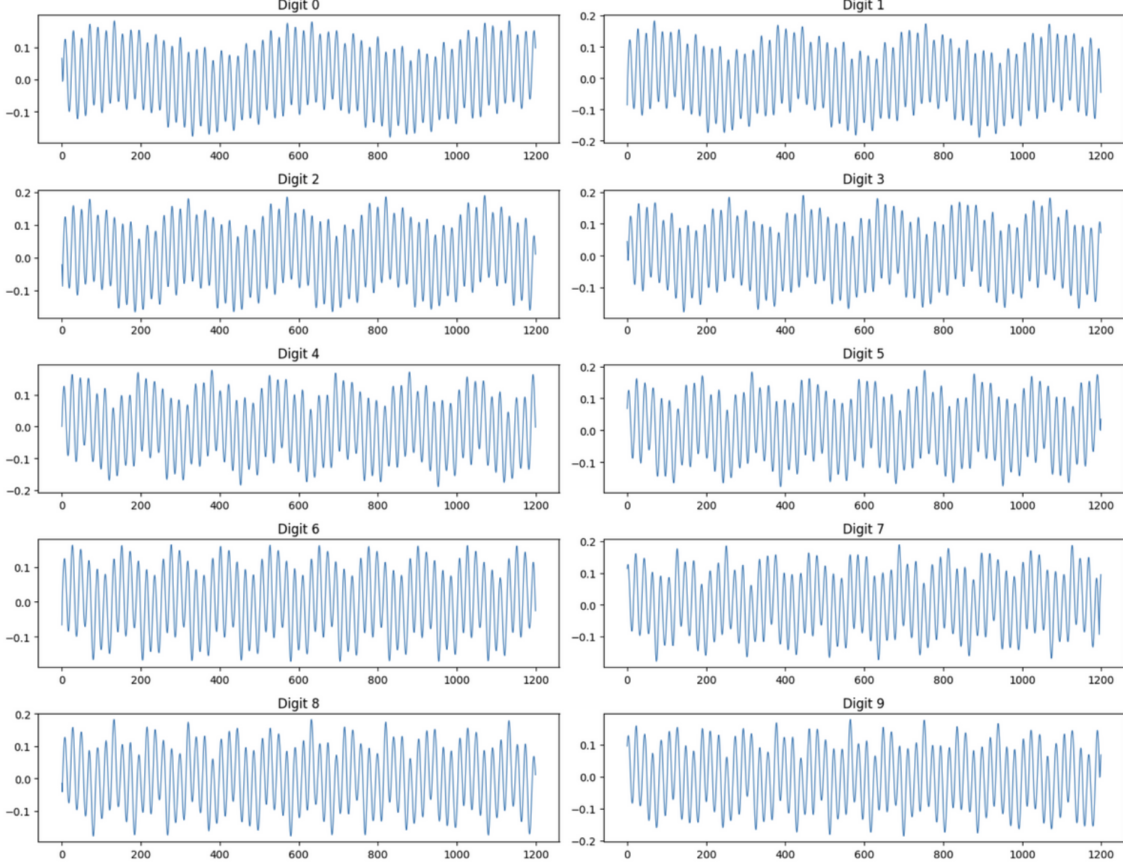


Figure 4: Representative power traces for digits 0 through 9. Each digit produces a distinct waveform due to differences in CPU instructions and data-dependent operations.

The result confirms that the simplified digital lock leaks enough information through its power consumption to fully reconstruct the passkey without interacting with the firmware or observing the code directly.

The segmentation and labeling approach is illustrated in Figure 3, and the waveform-level matches are shown in Figure 5.

Discussion

Practical Considerations

Although the experiment demonstrates a successful attack, several factors simplify the scenario compared to real-world systems:

- The device executes a relatively simple, deterministic routine for each digit, making traces highly repeatable.
- Environmental noise is limited; the bench setup uses a stable power supply and short wiring.
- The attacker has full control over the timing and input digits, which simplifies alignment.

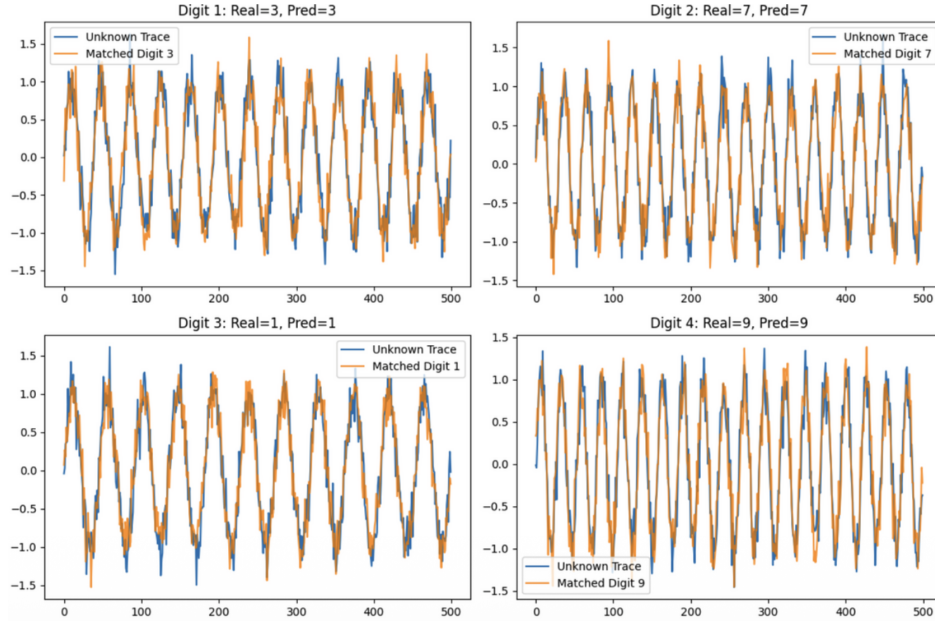


Figure 5: Comparison between unknown digit traces (blue) and their matched templates (orange). In this sample experiment, the recovered passkey is 3–7–1–9.

In more complex devices, additional preprocessing steps—such as filtering, resampling, and advanced alignment techniques—would be required. Moreover, cryptographic algorithms often involve masking and other countermeasures that reduce leakage.

Ethical and Security Implications

This project highlights how even low-cost microcontrollers can be vulnerable to side-channel attacks if physical access is granted. Manufacturers of secure tokens, smartcards, and IoT devices must consider these threats during the design phase. From an educational standpoint, the experiment is valuable because it makes an abstract security concept tangible and measurable on the lab bench.

Conclusion

This project demonstrates a complete workflow for performing a power-analysis side-channel attack on an Arduino-based digital lock. By inserting a shunt resistor into the power path, capturing high-resolution waveforms with a digital oscilloscope, and applying basic template matching in Python, it is possible to reconstruct a secret four-digit passkey from power measurements alone.

The experiment reinforces an important lesson for hardware and embedded-system designers: security is not only a matter of cryptographic algorithms, but also of physical implementation. Future work could extend this platform to more realistic cryptographic primitives, explore automated trace alignment and feature extraction, and evaluate the effectiveness of different countermeasures on the same hardware.